



**National University of Computer and Emerging Sciences  
FAST School of Computing, Lahore Campus**

## **Life at FAST**

Course: Software Construction and Development (SE3005)

Program: BS (Software Engineering)

Semester: Fall 2026

Assignment: Assignment 1

Material: [https://drive.google.com/drive/folders/1SdNQDctjxp\\_rEulYg5CWe9m\\_dm5kyO0](https://drive.google.com/drive/folders/1SdNQDctjxp_rEulYg5CWe9m_dm5kyO0)

## Instructions

1. **Submit the GitHub repository link** as your assignment submission.
2. **Plagiarism / Cheating Policy**
  - You are **not allowed to copy solutions from other students**.
  - Your code will be checked using plagiarism detection tools.
  - If any form of cheating or plagiarism is found, **heavy penalties will be given to all students involved**.
3. **Late Submission Policy**
  - **Late submissions will not be accepted**.
  - Use of **GitHub is mandatory**.
  - Any commit made **after the submission deadline will not be considered valid**.
4. **GitHub Collaborator**
  - Add us as a **collaborator** to your **private** repository.
  - **Section 5A:** thomas-catt
  - **Sections 5B & 9A:** m-ahmad-butt
5. **Submission**
  - Submit **only the GitHub repository link** through the designated submission form.
6. **Diagrams**
  - If the diagrams in the assignment PDF appear blurry, you can download the **high-resolution diagrams from the provided Google Drive link**.
7. **Vibe Coding**
  - **Vibe coding is strictly prohibited**.
  - You are expected to understand and be able to explain the code you submit.
  - Using AI tools to generate the complete solution without understanding or implementing it yourself is not allowed.
8. **File Handling & Logging**
  - **All application data must be stored in files**. Students should not rely solely on hardcoded data or in-memory collections for persistent data.
  - All .txt files used by the application must be placed inside the **data/** folder.
  - When the application is executed, it must generate and maintain application logs inside the **logs/** folder.
  - The log file should record important application activities and events, such as registrations, course operations, assignment operations, request processing, errors, etc.

# 1. Introduction: Campus Life at FAST

The **Campus Management System** is designed to support the day-to-day academic activities of students, instructors, and university staff at a campus such as FAST National University. The system focuses on core activities performed throughout a typical semester, including course registration, attendance tracking, assignment creation and evaluation, and handling academic requests.

At the start of a semester, the **Academic Office Admin** creates courses and sections, assigns instructors, and sets section capacities and rooms. Once sections are established, a **Permanent Instructor** can assign a normal student as a Teaching Assistant (TA) for the section. Students can then browse available courses, check their credit hours, and register for sections that fit their timetable. At the time of registration, the system calculates the student's total registered credit hours. Once classes begin, both visiting and permanent instructors manage student attendance, while TAs create assignments and evaluate the work submitted by students.

The system also provides an **academic request management** facility. Students can submit requests related to course or section clashes, grading, teaching, professors, classmates, or other academic concerns. The **Academic Office Admin** can view, process, approve, or reject these requests.

The main actors covered by the system are **Normal Student, Teaching Assistant, Visiting Instructor, Permanent Instructor, and Academic Office Admin**.

The remainder of this document presents representative user scenarios for these main actors, followed by the class diagrams that define the structure of the system and the use case diagrams that summarize the functionality available to each actor.

## 2. User Flows: Scenario Based

### **Ahmad Butt (*Teaching Assistant*)**

He is working as a Teaching Assistant for a section. He can view his assigned section and the students enrolled in it. He creates an assignment, sets its deadline and total marks, and once students submit their work, he can view the submissions, check late submissions, evaluate them, assign marks, and provide feedback.

### **Saim Arif (*Normal Student*)**

He wants to register his courses for the semester. He views the available courses along with their credit hours, selects the required sections, and registers for them. The system calculates his total registered credit hours at the time of registration. He can then view his registered courses and timetable, check his assignments and submit them, view his attendance and attendance percentage, and submit a course clash request if needed.

### **Arslan Asif (*Permanent Instructor*)**

He teaches a section and manages everything related to his students. He views his assigned courses, sections, and enrolled students, then marks attendance as Present, Absent, or Late and can update it when required. He can also calculate students' attendance percentages. As a Permanent Instructor, Arslan can assign a Normal Student as a Teaching Assistant for his section. He also supervises Final Year Project groups, views their details and members, schedules meetings, and evaluates project ideas with feedback.

### **Faizan (*Visiting Instructor*)**

He is a Visiting Instructor who teaches his assigned section. He views his courses, sections, and enrolled students and manages their attendance by marking them Present, Absent, or Late. He can update attendance and calculate attendance percentages when required. Unlike a Permanent Instructor, he cannot assign a Teaching Assistant and took FYP supervision.

### **Fateh (*Academic Office Admin*)**

He manages the academic structure of the university. He creates, updates, and searches courses and creates and updates sections for those courses. He can set section capacities, assign rooms, and assign instructors to sections. He also manages academic requests by viewing, approving, or rejecting submitted requests.

### 3. Class Diagrams

#### 3.1 Person and Role Hierarchy

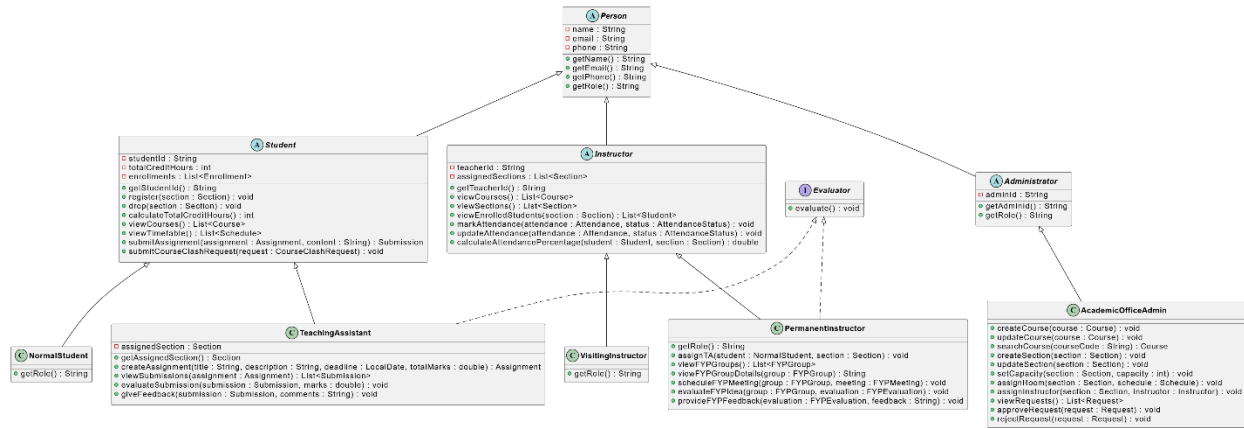


Figure 1. Person and Role Hierarchy

### 3.2 Course, Section, Enrollment, Attendance and Schedule

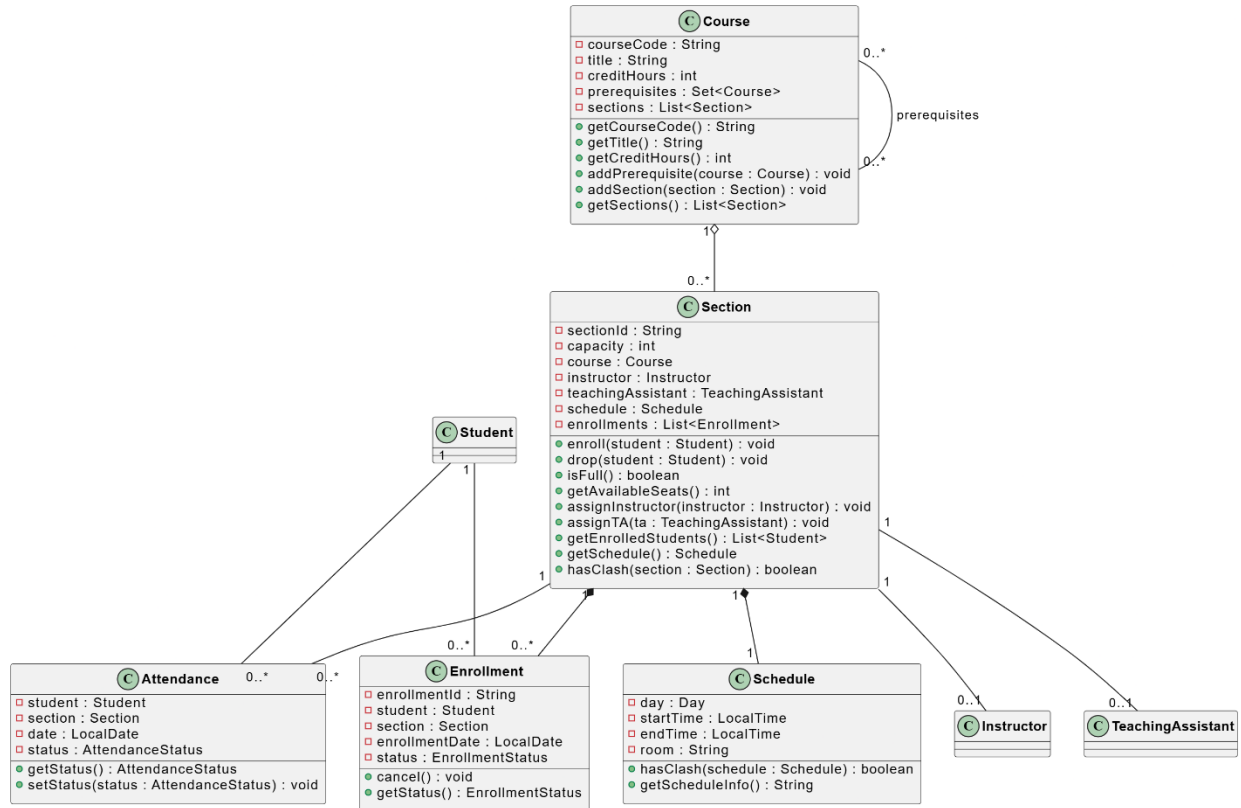


Figure 2. Course, Section, Enrollment, Attendance and Schedule

### 3.3 Request and Section Request

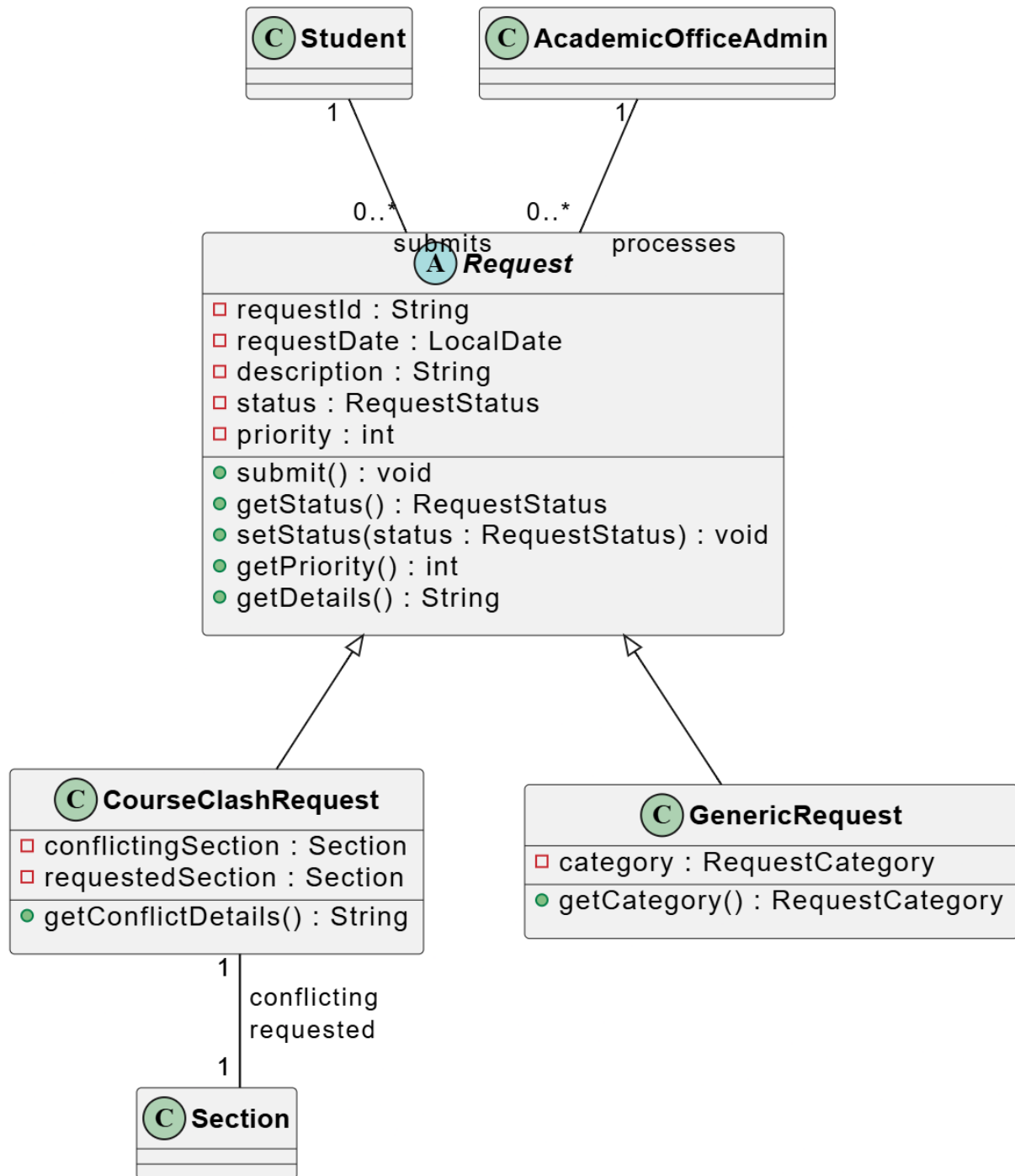


Figure 3. Request and Section Request

### 3.4 Assessment, Assignment, Submission and Feedback

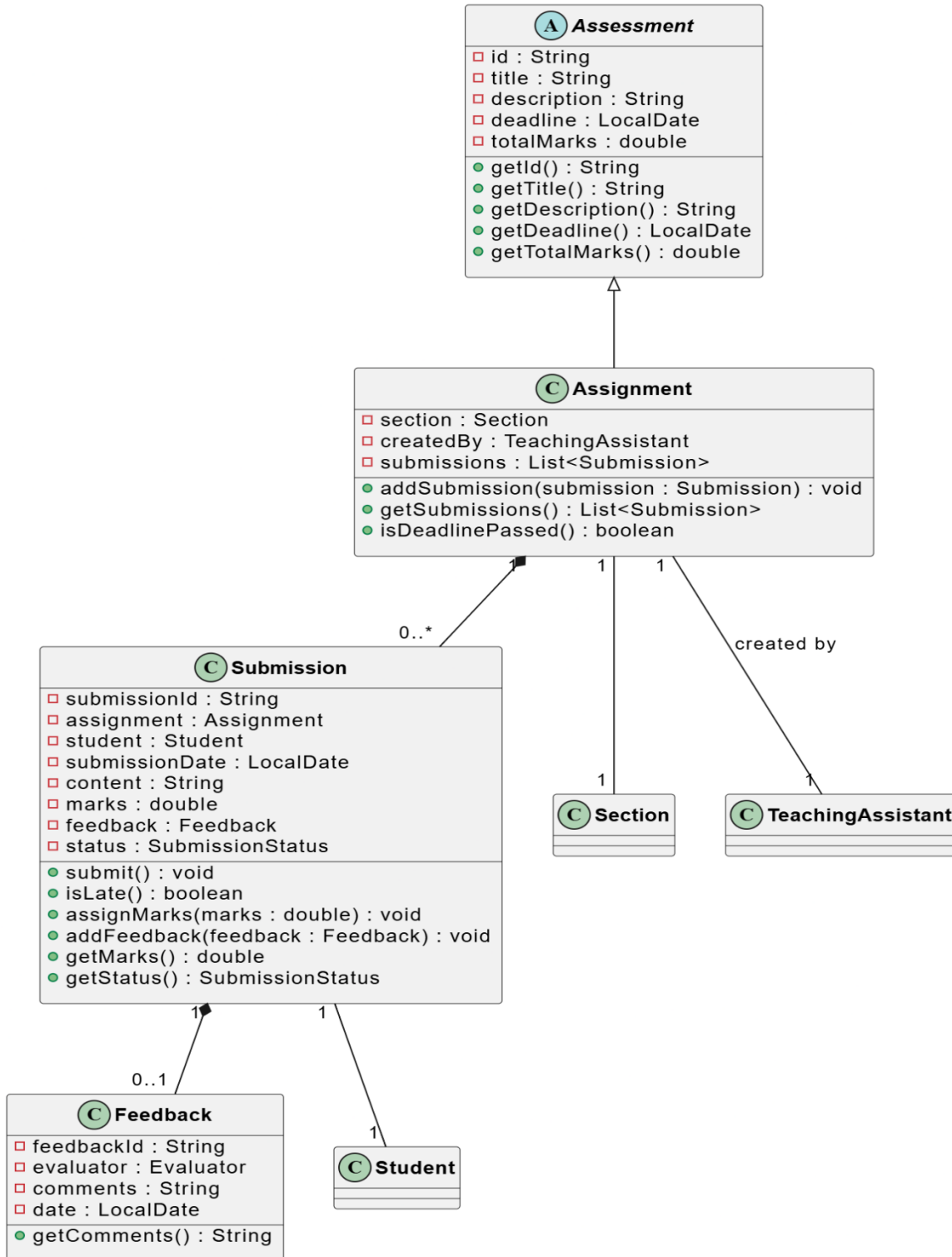


Figure 4. Assessment, Assignment, Submission and Feedback

### 3.6 Final Year Project

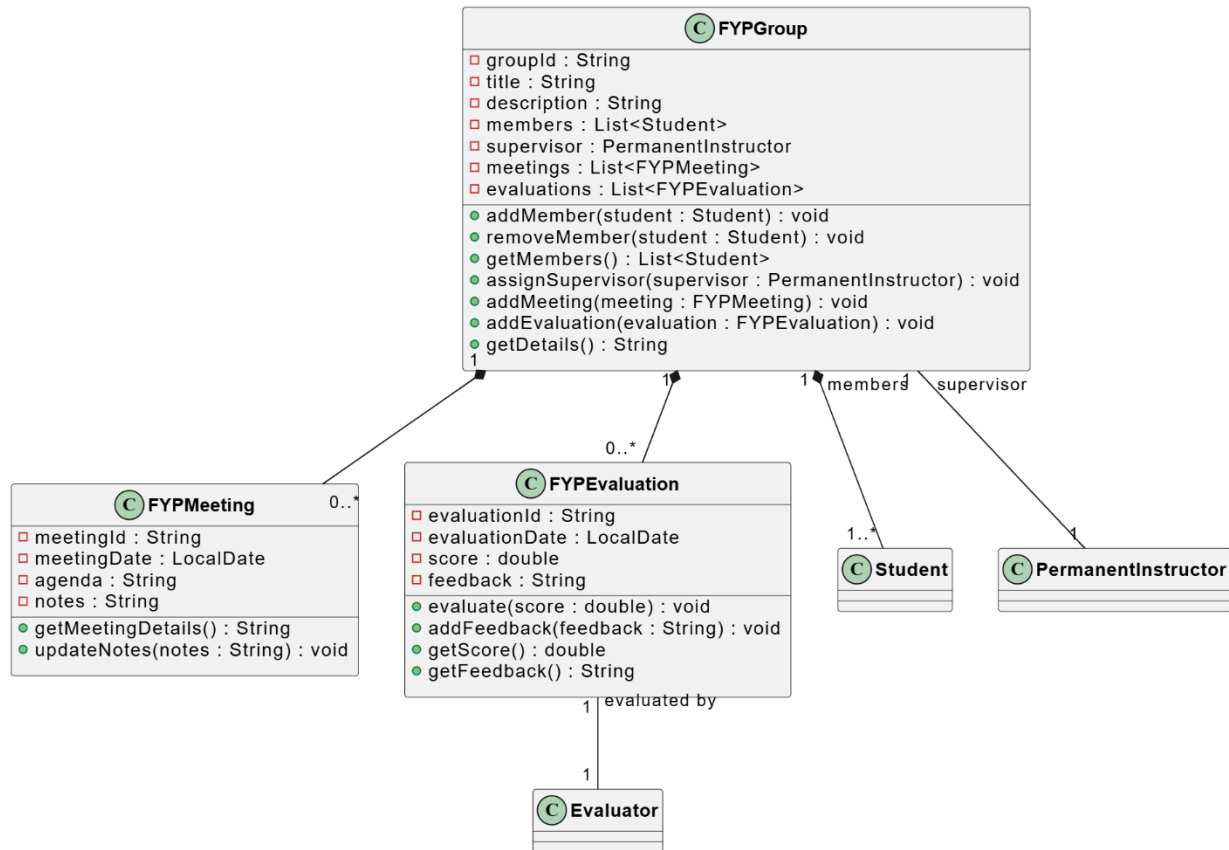


Figure 6. Final Year Project and Sports Item

### 3.7 Enumerations and Comparators

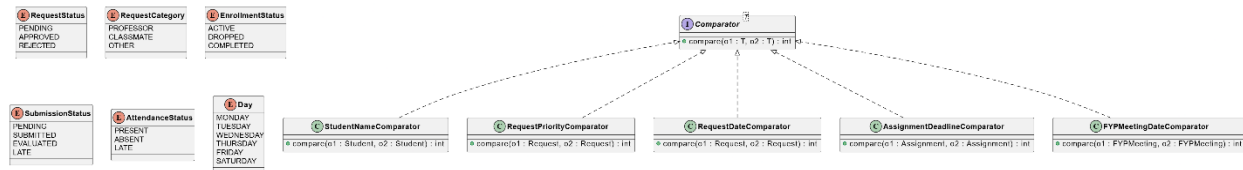


Figure 7. Enumerations and Comparators

### 3.8 Exception Hierarchy

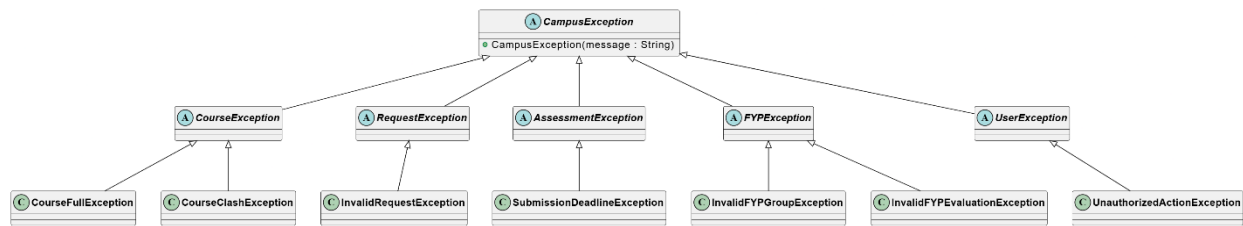
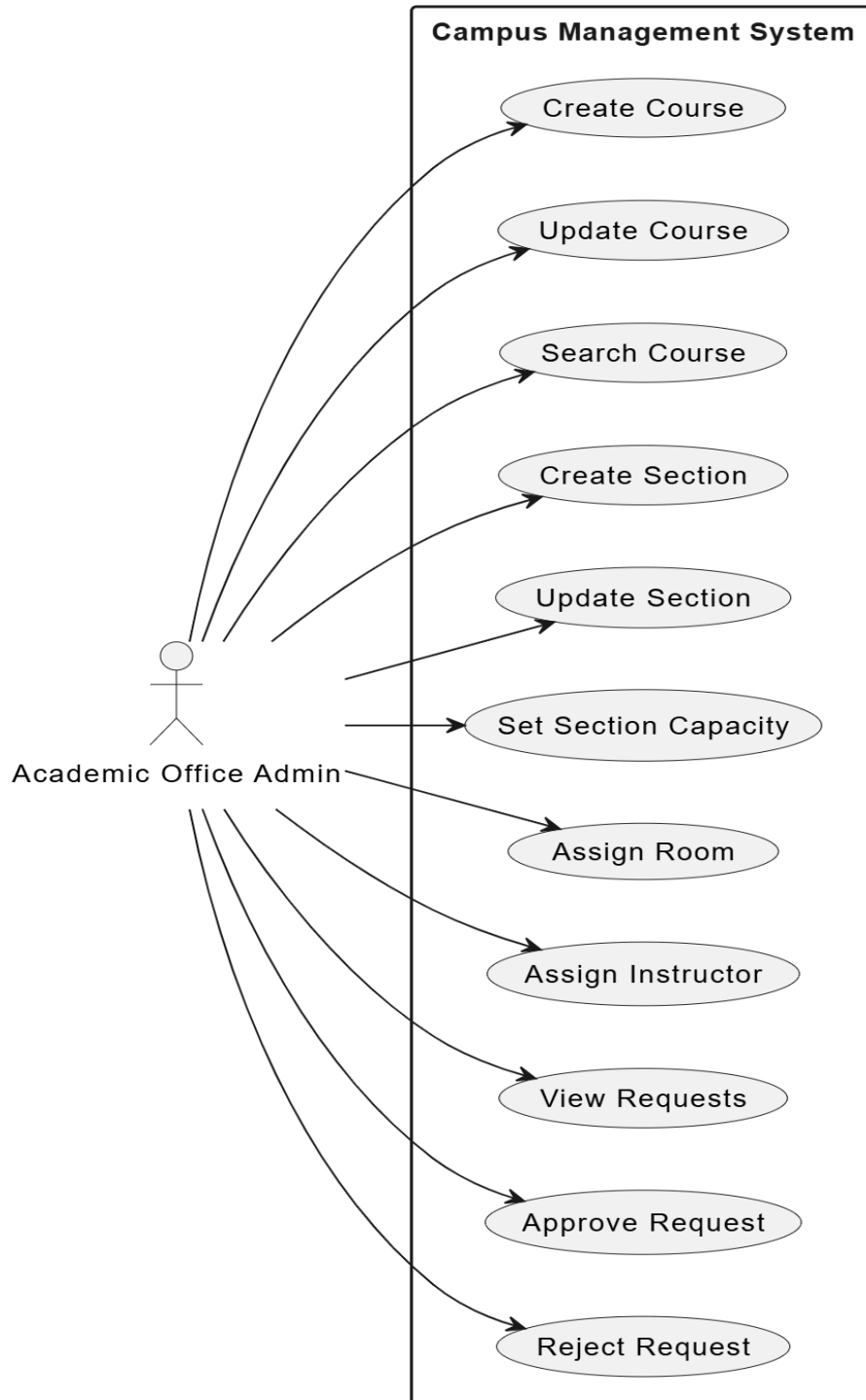


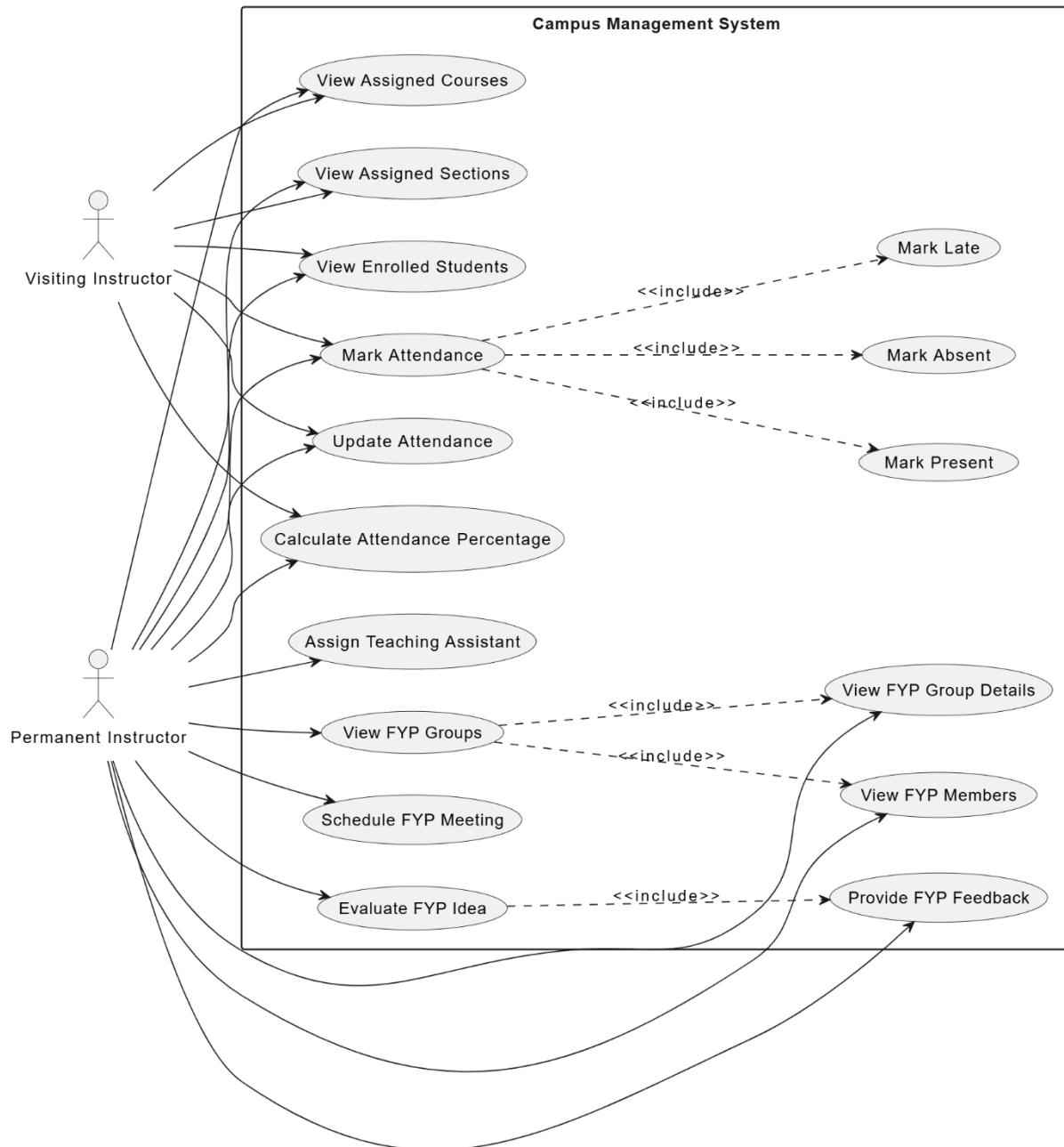
Figure 8. Exception Hierarchy

## 4. Use Case Diagrams

### 4.1 Administrator Use Cases



## 4.2 Instructor Use Cases



## 4.3 Student Use Cases

